

CHAPTER 10 • INFRASTRUCTURE AS CODE

Terraform Loops & Conditionals

Building dynamic, DRY infrastructure with count, for_each, dynamic blocks, and conditional expressions.



What we'll cover

Terraform gives us three core tools to keep infrastructure code short, safe, and reusable.



10.1

count vs. for_each

Two looping mechanisms for creating multiple resources.



10.2

Dynamic blocks

Remove repetition inside nested configuration blocks.



10.3

Conditional expressions

The ternary operator for environment-aware config.



10.4

Scenario walkthrough

Deploying EC2 instances dynamically, end to end.

Two ways to loop over resources

Feature	count	for_each
Type	Integer for iteration	Map or set for iteration
Best for	Many similar resources	Unique, keyed resources
Key-based indexing	No — uses index numbers	Yes — uses map/set keys
Removing an item	Can cause reordering issues	No reordering issues



Rule of thumb: reach for **count** for identical, throwaway copies; reach for **for_each** whenever each resource needs its own stable identity.

Create 3 EC2 instances with count

```
resource "aws_instance" "example" {  
  count      = 3  
  ami       = "ami-12345678"  
  instance_type = "t2.micro"  
  tags = {  
    Name = "server-${count.index}"  
  }  
}
```



Result

Creates 3 instances: server-0, server-1, server-2.



Issue

Removing an instance can shift index positions, forcing Terraform to recreate resources unexpectedly.

Use count for identical, interchangeable resources where order doesn't carry meaning.

Create instances dynamically with for_each

```
variable "instances" {  
  type = map(string)  
  default = {  
    "web" = "t2.micro"  
    "app" = "t3.small"  
    "db" = "t3.medium"  
  }  
}
```

```
resource "aws_instance" "example" {  
  for_each      = var.instances  
  ami           = "ami-12345678"  
  instance_type = each.value  
  tags = {  
    Name = each.key  
  }  
}
```



Creates web, app, and db instances with unique, stable names — removing "db" later never touches web or app.

Generate nested blocks without repetition

```
variable "ingress_rules" {
  default = [
    { port = 80, protocol = "tcp" },
    { port = 443, protocol = "tcp" }
  ]
}

resource "aws_security_group" "example" {
  name = "dynamic-sg"

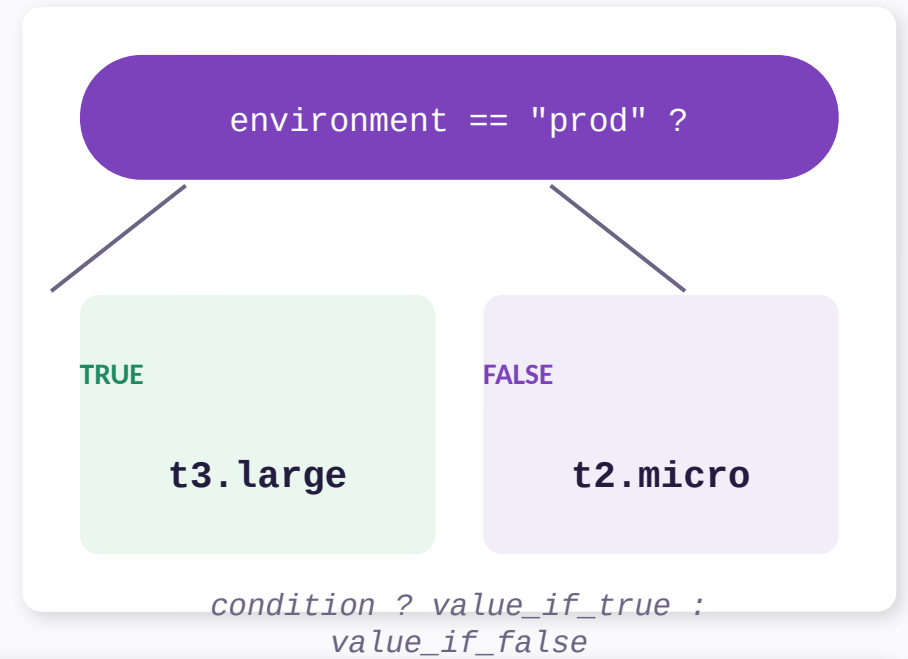
  dynamic "ingress" {
    for_each = var.ingress_rules
    content {
      from_port   = ingress.value.port
      to_port     = ingress.value.port
      protocol    = ingress.value.protocol
      cidr_blocks = ["0.0.0.0/0"]
    }
  }
}
```

Why it matters

- for_each iterates the ingress_rules list
- content { } is repeated once per rule
- Adds ports 80 & 443 automatically
- No copy-pasted ingress blocks

The ternary operator: ? :

```
variable "environment" {  
  default = "dev"  
}  
  
resource "aws_instance" "example" {  
  ami          = "ami-12345678"  
  instance_type = var.environment == "prod" ?  
    "t3.large" : "t2.micro"  
}
```



In plain English: if **environment** = "prod", assign **"t3.large"** — otherwise fall back to **"t2.micro"**.

Deploying EC2 instances dynamically

Step 1 of 2 — Define a typed variable for per-instance settings

1

2

```
variable "instances" {  
  type = map(object({  
    instance_type = string  
    volume_size   = number  
  }))  
  default = {  
    "web" = { instance_type = "t2.micro", volume_size = 10 }  
    "app" = { instance_type = "t3.small", volume_size = 20 }  
  }  
}
```


Deploying EC2 instances dynamically

Step 2 of 2 — Wire the variable into a for_each resource

1

```
resource "aws_instance" "example" {  
  for_each      = var.instances  
  ami           = "ami-12345678"  
  instance_type = each.value.instance_type  
  
  root_block_device {  
    volume_size = each.value.volume_size  
  }  
  
  tags = { Name = each.key }  
}
```

2

Result: 2 instances



web

t2.micro • 10 GB volume



app

t3.small • 20 GB volume

Dynamic configuration, recapped



count vs. for_each

Loop over resources by index or by stable key.



Conditional expressions

condition ? true_val : false_val for environment logic.



Dynamic blocks

Repeat nested blocks like ingress rules without copy-paste.



Scenario

for_each + typed objects to provision varied EC2 instances.